```
# Python Basics: Lists, Dictionaries, & Booleans
Writing and running Python code
Basic Python objects, including lists and dictionaries
Changing values and combining objects
Comparing values with booleans
```

IN [39]
```
a=6
b=3
a/b
```

OUT [39]
```
2.0
```

IN [41]
```
'this is a string'
'james brown'
```

OUT [41]
```
'james brown'
```

IN [16]
```
first_string = 'hello from world'
(first_string)
```

OUT [16]
```
'hello from world'
```

IN [31]
```
second_string = "one bacon double cheeseburger"
second_string
```

OUT [31]
```
'one bacon double cheeseburger'
```

IN [1]
```
cities = ['Tokyo','Los Angeles','New York','San Francisco']
print(cities[1])
cities[0]
```

OUT [1]
```
'Tokyo'
```

Los Angeles

IN [48]

```
city_population = {
    'Tokyo': 13350000, # a key-value pair
    'Los Angeles': 18550000,
    'New York City': 8400000,
    'San Francisco': 1837442,
}
city_population
city_population ['Tokyo']
```

OUT [48]

13350000

IN [52]

```
city_population['Los Angeles']= 1000000
city_population['Scotland']= 50000
city_population
```

OUT [52]

```
{'Tokyo': 13350000,
 'Los Angeles': 1000000,
 'New York City': 8400000,
 'San Francisco': 1837442,
 'Scotland': 50000}
```

IN [55]

```
municipalities = {
    'New York City': [
        'Manhattan',
        'The Bronx',
        'Brooklyn',
        'Queens',
        'Staten Island'
    ],
    'Tokyo': [
        'Akihabara',
        'Harajuku',
        'Shimokitazawa',
        'Nakameguro',
        'Shibuya',
        'Ebisu/Daikanyama',
        'Shibuya District',
        'Aoyama',
```

```
        'Asakusa/Ueno',
        'Bunkyo District',
        'Ginza',
        'Ikebukuro',
        'Koto District',
        'Meguro District',
        'Minato District',
        'Roppongi',
        'Shinagawa District',
        'Shinjuku',
        'Shinjuku District',
        'Sumida District',
        'Tsukiji',
        'Tsukishima']
}
municipalities['Tokyo'][3]

```

OUT [55] 'Nakameguro'

IN [67]
```
type(municipalities['Tokyo'])

```

OUT [67] list

IN [69]
```
city_population['Tokyo'] == 13350000
city_population['Tokyo'] != 50000

```

OUT [69] True

```
# Python Methods, Functions, & Libraries
Methods of dictionaries, specifically .keys() and .values()
Functions, specifically print, type(), and len()
Importing libraries with import
```

IN [60]

```
city_population = {
  'Tokyo': 13350000, # a key-value pair
  'Los Angeles': 18550000,
  'New York City': 8400000,
  'San Francisco': 1837442,
}
```

IN [93]

```
list(city_population.keys())
```

OUT [93]

```
['Tokyo', 'Los Angeles', 'New York City', 'San Francisco']
```

IN [82]

```
print (type(city_population.keys()))
print (type(city_population.values()))
```

OUT [82]

```
<class 'dict_keys'>
<class 'dict_values'>
```

IN [58]

```
list(city_population.keys())[2]
```

OUT [58]

```
'New York City'
```

IN [43]

```
(city_population.values())
```

OUT [43]

```
dict_values([13350000, 18550000, 8400000, 1837442])
```

IN [42]

```
1 city_population['Los Angeles']
```

OUT [42]

```
18550000
```

IN [34]

```
municipalities = {
    'New York City': [
        'Manhattan',
        'The Bronx',
        'Brooklyn',
        'Queens',
        'Staten Island'
    ],
    'Tokyo': [
        'Akihabara',
        'Harajuku',
        'Shimokitazawa',
        'Nakameguro',
        'Shibuya',
        'Ebisu/Daikanyama',
        'Shibuya District',
        'Aoyama',
        'Asakusa/Ueno',
        'Bunkyo District',
        'Ginza',
        'Ikebukuro',
        'Koto District',
        'Meguro District',
        'Minato District',
        'Roppongi',
        'Shinagawa District',
        'Shinjuku',
        'Shinjuku District',
        'Sumida District',
        'Taukiji'
```

```
        'Tsukiji',
        'Tsukishima']
}
```

IN [50]
```
municipalities['Tokyo'][3]
```

OUT [50]
```
'Nakameguro'
```

IN [51]
```
municipalities.keys()
```

OUT [51]
```
dict_keys(['New York City', 'Tokyo'])
```

IN [38]
```
list(municipalities.values())
```

OUT [38]
```
[['Manhattan', 'The Bronx', 'Brooklyn', 'Queens', 'Staten Island'],
 ['Akihabara',
  'Harajuku',
  'Shimokitazawa',
  'Nakameguro',
  'Shibuya',
  'Ebisu/Daikanyama',
  'Shibuya District',
  'Aoyama',
  'Asakusa/Ueno',
  'Bunkyo District',
  'Ginza',
  'Ikebukuro',
  'Koto District',
  'Meguro District',
  'Minato District',
  'Roppongi',
  'Shinagawa District',
  'Shinjuku',
  'Shinjuku District',
```

```
'Sumida District',
'Tsukiji',
'Tsukishima']]
```

IN [81]

```
print (type(municipalities))
print (type(municipalities['Tokyo']))
print (type(municipalities['Tokyo'][0]))
```

OUT [81]

```
<class 'dict'>
<class 'list'>
<class 'str'>
```

IN [24]

```
print (list(city_population.values()))
print (list(city_population.values())[2])
print (type(list(city_population.values())[0]))
```

OUT [24]

```
[13350000, 18550000, 8400000, 1837442]
8400000
<class 'int'>
```

IN [39]

```
len(municipalities['Tokyo'])
```

OUT [39]

```
22
```

IN [43]

```
print(municipalities['Tokyo'][2])
len(municipalities['Tokyo'][2])
```

OUT [43]

```
13
```

Shimokitazawa

IN [55]

```
print(len(municipalities))
len(city_population)
```

OUT [55]

```
4
```

```
2
```

IN [48]

```
import numpy
```

IN [64]

```
population_values = city_population.values()
list(population_values)
```

OUT [64]

```
[13350000, 18550000, 8400000, 1837442]
```

IN [65]

```
numpy.mean(list(population_values))
```

OUT [65]

```
np.float64(10534360.5)
```

```
# Creating Pandas DataFrames & Selecting Data
Describe a pandas DataFrame
Create a pandas DataFrame with data
Select columns in a DataFrame
Select rows in a DataFrame
Select both columns and rows in a DataFrame
```

IN [1]
```
datasets[0].head(n=5)
```

OUT [1]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_secti
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google	No
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	te
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-car
3	None	2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O	None	No
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org	No

IN [3]
```
import pandas as pd
```

IN [4]
```
data = datasets[0]
```

IN [5]
```
data = data.fillna('')
```

IN [11]

```
print (data['url'].head(n=10))
type(data['url'].head(n=10))
```

OUT [11]

```
pandas.core.series.Series


0                                https://watsi.org/
1             https://watsi.org/team/the-meteor-chef
2                       https://watsi.org/gift-cards
3                                https://watsi.org/
4                                https://watsi.org/
5                   https://watsi.org/fund-treatments
6                             https://watsi.org/redeem
7    https://watsi.org/profile/4f0c9b3fb0b1-rasty-noel
8                   https://watsi.org/fund-treatments
9                                https://watsi.org/
Name: url, dtype: object
```

IN [16]

```
data[:3]
```

OUT [16]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google		Desktop
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	team	Desktop
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-cards	Desktop

IN [17]

```
data[3:]
```

OUT [17]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
3		2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O			Desktop
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org		Desktop
5	http://blog.watsi.org/	2015-10-15 06:04:40+00:00	Watsi | Fund a medical treatment on Watsi. 100...	https://watsi.org/fund-treatments	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	SHAREN Y	watsi.org	fund-treatments	Desktop
6		2015-12-25 10:23:43+00:00	Watsi | Redeem your Watsi Gift Card	https://watsi.org/redeem	Mozilla/5.0 (Windows NT 6.1; WOW64; rv:43.0) G...	BRICE Z		redeem	Desktop
7	https://watsi.org/team/watsi	2015-11-04 23:09:54+00:00	Watsi | Rasty Noel is a four-year-old boy from...	https://watsi.org/profile/4f0c9b3fb0b1-rasty-noel	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	TAMICA R	watsi.org	profile	Desktop
...	...	...	...	...	...	...	...	...	...
4995	https://watsi.org/	2015-11-02 22:23:31+00:00	Watsi | Fund a medical treatment on	https://watsi.org/fund-treatments	Mozilla/5.0 (Macintosh; Intel Mac OS X	MADONNA Z	watsi.org	fund-treatments	Desktop

IN [22]

```
print(data.iloc[0])
data.iloc[1]
```

OUT [22]

```
referrer           https://themeteorchef.com/snippets/making-use-...
timestamp                                  2016-02-24 23:12:10+00:00
title                                        Watsi | The Meteor Chef
url                          https://watsi.org/team/the-meteor-chef
user_agent         Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...
user_id                                                     WARREN Q
referrer_domain                                       themeteorchef.com
website_section                                                  team
platform                                                      Desktop
Name: 1, dtype: object


referrer                                       https://www.google.com/
timestamp                                  2016-02-05 00:48:23+00:00
title              Watsi | Fund medical treatments for people aro...
url                                                  https://watsi.org/
user_agent         Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...
user_id                                                   CHAROLETTE S
referrer_domain                                                 google
website_section
platform                                                      Desktop
Name: 0, dtype: object
```

IN [31]

```
1  data['url'].iloc[3]
2  data['url'][:2]
```

OUT [31]

```
0                       https://watsi.org/
1    https://watsi.org/team/the-meteor-chef
Name: url, dtype: object
```

IN [32]

```
1  data['referrer'][10:15]
```

OUT [32]

```
10
11                        https://www.facebook.com/
12    https://watsi.org/profile/ef9cce891dc4-anibal
13
14          https://watsi.org/donor/settings/payment
Name: referrer, dtype: object
```

Display Table

Md

```
andas .values_count() & .plot()
Analyze and visualize real web traffic data using a DataFrame
Count the number of times a value occurs using .values_count()
Plot bar charts with .plot()
```

IN [1]

Py

```
datasets[0].head(n=5)
import pandas as pd
```

IN [9]

Py

```
data=datasets[0]
data = data.fillna('suck it')
```

IN [10]

Py

```
data.head()
```

OUT [10]

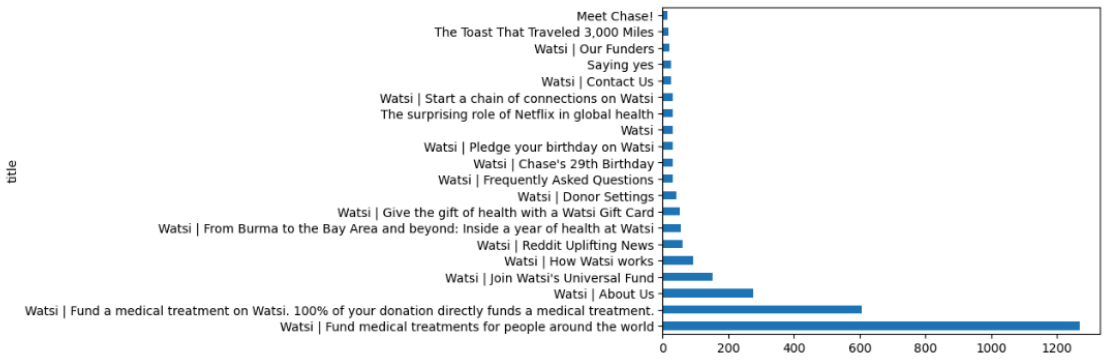
index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google	suck it	Desktop
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	team	Desktop
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-cards	Desktop
3	suck it	2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O	suck it	suck it	Desktop
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org	suck it	Desktop

IN [12]
```
data['title'].value_counts()[:20]
```

OUT [12]
```
title
Watsi | Fund medical treatments for people around the world                                1268
Watsi | Fund a medical treatment on Watsi. 100% of your donation directly funds a medical treatment.     606
Watsi | About Us                                                                           276
Watsi | Join Watsi's Universal Fund                                                        151
Watsi | How Watsi works                                                                    93
Watsi | Reddit Uplifting News                                                              61
Watsi | From Burma to the Bay Area and beyond: Inside a year of health at Watsi           55
Watsi | Give the gift of health with a Watsi Gift Card                                    52
Watsi | Donor Settings                                                                     41
Watsi | Frequently Asked Questions                                                        32
Watsi | Chase's 29th Birthday                                                             32
Watsi | Pledge your birthday on Watsi                                                     32
Watsi                                                                                     32
The surprising role of Netflix in global health                                           30
Watsi | Start a chain of connections on Watsi                                           30
Watsi | Contact Us                                                                        27
Saying yes                                                                                25
Watsi | Our Funders                                                                       20
The Toast That Traveled 3,000 Miles                                                       17
Meet Chase!                                                                               16
Name: count, dtype: int64
```

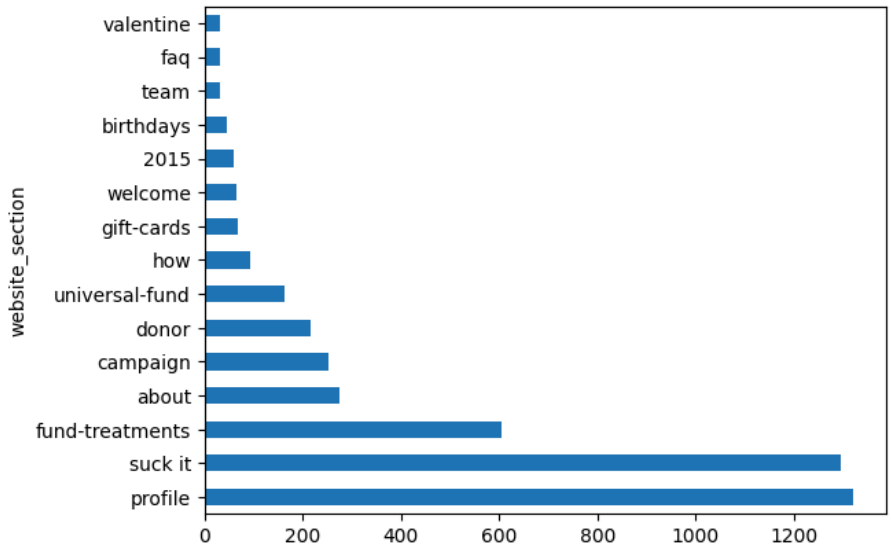
IN [16]
```
data['title'].value_counts()[:20].plot(kind='barh')
```

OUT [16]
```
<Axes: ylabel='title'>
```



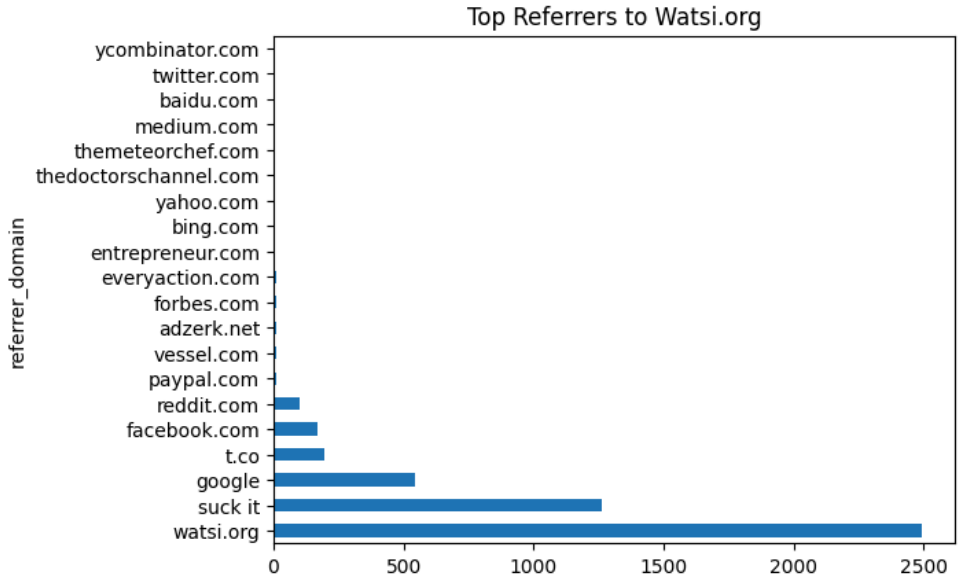
IN [29]
```
1 data['website_section'].value_counts()[:15].plot(kind='barh')
```

OUT [29]
```
<Axes: ylabel='website_section'>
```



IN [24]
```
1 data['referrer_domain'].value_counts()[:20]
2 data['referrer_domain'].value_counts()[:20].plot(kind='barh', title='Top Referrers to Watsi.org')
```

OUT [24]
```
<Axes: title={'center': 'Top Referrers to Watsi.org'}, ylabel='referrer_domain'>
```

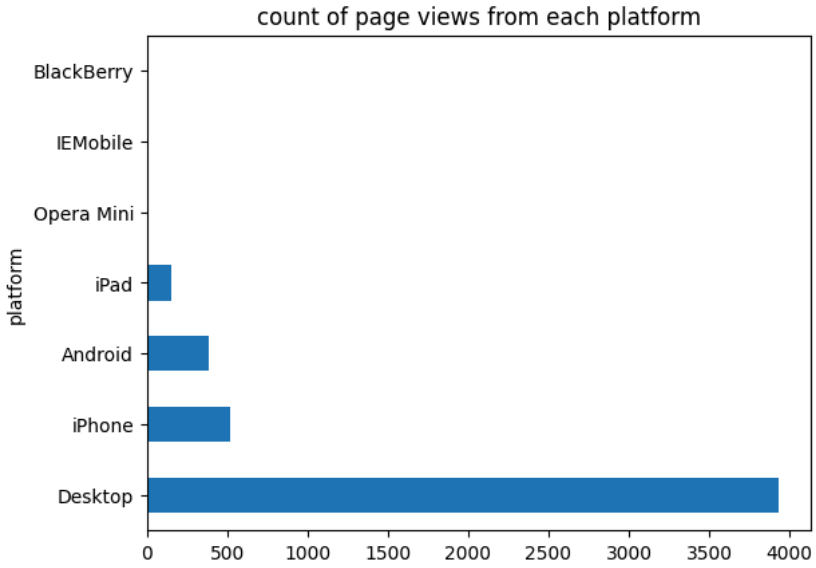


IN [25]

```
1 data['platform'].value_counts()[:15].plot(kind='barh',title='count of page views from each platform')
```

OUT [25]

```
<Axes: title={'center': 'count of page views from each platform'}, ylabel='platform'>
```



Md

```
# Filtering Data in Python with Boolean Indexes
Filter and segment data using boolean indexing
Partially match text with .str.contains()
```

IN [4]

Py

```
import pandas as pd
```

IN [3]

Py

```
data=datasets[0]
data=data.fillna('')
data.head()
```

OUT [3]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google		Desktop
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	team	Desktop
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-cards	Desktop
3		2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O			Desktop
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org		Desktop

IN [47]

```
data['title']=='Watsi | Fund medical treatments for people around the world'
homepage_index=data['title']=='Watsi | Fund medical treatments for people around the world'
watsi_homepage=data[homepage_index]
watsi_homepage
watsi_homepage['referrer'].value_counts()[:5]
```

OUT [47]

```
referrer
                            451
https://www.google.com/     153
https://www.google.co.in/    53
https://watsi.org/           22
https://www.reddit.com/      21
Name: count, dtype: int64
```

IN [12]

```
#another way to do above cell
data[data['title'] == 'Watsi | Fund medical treatments for people around the world']
my_way=data[data['title'] == 'Watsi | Fund medical treatments for people around the world']
my_way['referrer'].value_counts()[:5]
```

OUT [12]

```
referrer
                            451
https://www.google.com/     153
https://www.google.co.in/    53
https://watsi.org/           22
https://www.reddit.com/      21
Name: count, dtype: int64
```

IN [13]

```
watsi_reddit_pageviews = data[data['referrer_domain'] == 'reddit.com']
watsi_reddit_pageviews['title'].value_counts()[:15]
```

OUT [13]

```
title
Watsi | Reddit Uplifting News                                  52
Watsi | Fund medical treatments for people around the world    36
The surprising role of Netflix in global health                 9
Watsi | Give the gift of health with a Watsi Gift Card          2
Name: count, dtype: int64
```

IN [25]

```
data['referrer'].str.contains('medical')
medical_referrer_index = data['referrer'].str.contains('medical')
medical_referrals = data[medical_referrer_index]
medical_referrals
```

OUT [25]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
222	http://www.inc.com/abigail-tracy/new-crowdfund...	2015-12-07 23:15:52+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (iPad; CPU OS 9_1 like Mac OS X) A...	BERNADETTE W	inc.com		iPad
568	http://www.google.pl/url?sa=t&source=web&cd=1&...	2016-03-19 10:03:28+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Linux; Android 5.1.1; SAMSUNG SM-...	KRISTIAN E	google		Android
3115	https://www.reddit.com/r/Anarcho_Capitalism/co...	2015-12-19 00:33:04+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	JAIMEE U	reddit.com	gift-cards	Desktop
3886	http://poormedicalcare.weebly.com/help-our-cau...	2016-03-19 18:30:13+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebK...	STEPANIE V	weebly.com		Desktop
3980	http://my.parallaxsearch.com/web?qs=crowd+fund...	2016-03-11 05:22:01+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	ELLIE U	parallaxsearch.com		Desktop
4859	http://www.google.co.in/search?site=&oq=crowdf...	2016-03-14 18:59:07+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Linux; U; Android 2.3.6; en-us; S...	MARRY D	google		Android

IN [26]

```
medical_referrals['referrer'].tolist()
```

OUT [26]

```
['http://www.inc.com/abigail-tracy/new-crowdfunding-site-for-medical-students.html',
 'http://www.google.pl/url?sa=t&source=web&cd=1&rct=j&q=medical%20treatment%20grants&ved=0ahUKEwjRo5DGv8zLAhUiS5oKHdOAC58QFggaMAA&url=https%3
 'https://www.reddit.com/r/Anarcho_Capitalism/comments/3xddbx/watsi_a_cool_new_charity_that_crowdfunds_medical/',
 'http://poormedicalcare.weebly.com/help-our-cause.html',
 'http://my.parallaxsearch.com/web?qs=crowd+funding+for+medical+treatment',
 'http://www.google.co.in/search?site=&oq=crowdfunding+for+me&aqs=mobile-gws-lite.0.0l5&q=crowdfunding+for+medical+treatment']
```

IN [51]

```Py
data['referrer'].str.contains('crowdfund')
crowdfund_referrer_index = data['referrer'].str.contains('crowdfund')
crowdfund_referrals = data[crowdfund_referrer_index]
crowdfund_referrals
crowdfund_referrals['referrer'].tolist()
crowdfund_referrals['referrer'].value_counts()

#another way
#crowdfund_referrer_index = data['referrer'].str.contains('crowdfund')
#data[crowdfund_referrer_index]['referrer'].tolist()
```

OUT [51]

```
referrer
http://www.inc.com/abigail-tracy/new-crowdfunding-site-for-medical-students.html                                    1
https://www.reddit.com/r/Anarcho_Capitalism/comments/3xddbx/watsi_a_cool_new_charity_that_crowdfunds_medical/       1
http://www.google.co.in/search?site=&oq=crowdfunding+for+me&aqs=mobile-gws-lite.0.0l5&q=crowdfunding+for+medical+treatment    1
Name: count, dtype: int64
```

IN [53]

```Py
data['user_agent'].str.contains('IEMobile')
user_agent_index = data['user_agent'].str.contains('IEMobile')
user_agent = data[user_agent_index]
user_agent['user_agent'].tolist()
user_agent['user_agent'].value_counts()
```

OUT [53]

```
user_agent
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; Microsoft; Lumia 535 Dual SIM) like iPh
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; NOKIA; Lumia 635) like iPhone OS 7_0_3
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; Microsoft; Lumia 532; Orange) like iPho
Mozilla/5.0 (compatible; MSIE 10.0; Windows Phone 8.0; Trident/6.0; IEMobile/10.0; ARM; Touch; Microsoft; Lumia 640 XL Dual SIM)
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; NOKIA; Lumia 630) like iPhone OS 7_0_3
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; NOKIA; Lumia 1520) like iPhone OS 7_0_3
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; NOKIA; Lumia 929) like iPhone OS 7_0_3
Mozilla/5.0 (Mobile; Windows Phone 8.1; Android 4.0; ARM; Trident/7.0; Touch; rv:11.0; IEMobile/11.0; Microsoft; Lumia 435 Dual SIM) like iPh
Name: count, dtype: int64
```

Md

```
# Deriving New Columns & Defining Python Functions
Derive new columns from existing data
Write and test functions using in, if, else, and elif
Define functions using parameters and arguments
Apply functions to DataFrames using .apply()
Select multiple columns
```

IN [30]

Py

```
import pandas as pd
data=datasets[0]
data=data.fillna('Unknown')
```

IN [31]

Py

```
data.head()
```

OUT [31]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google	Unknown	Desktop
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	team	Desktop
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-cards	Desktop
3	Unknown	2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O	Unknown	Unknown	Desktop
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org	Unknown	Desktop

IN [32]

```
data['platform'].value_counts()
```

OUT [32]

```
platform
Desktop       3934
iPhone         515
Android        388
iPad           154
Opera Mini       7
IEMobile         1
BlackBerry       1
Name: count, dtype: int64
```

IN [33]

```
data['New']=2
data.head()
```

OUT [33]

index	referrer	timestamp	title	url	user_agent	user_id	referrer_domain	website_section	platform	New
0	https://www.google.com/	2016-02-05 00:48:23+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4...	CHAROLETTE S	google	Unknown	Desktop	2
1	https://themeteorchef.com/snippets/making-use-...	2016-02-24 23:12:10+00:00	Watsi | The Meteor Chef	https://watsi.org/team/the-meteor-chef	Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKi...	WARREN Q	themeteorchef.com	team	Desktop	2
2	https://watsi.org/	2015-12-25 17:59:35+00:00	Watsi | Give the gift of health with a Watsi G...	https://watsi.org/gift-cards	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_1...	MITCHEL O	watsi.org	gift-cards	Desktop	2
3	Unknown	2016-02-05 21:19:30+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_2...	MICHEL O	Unknown	Unknown	Desktop	2
4	https://watsi.org/fund-treatments	2016-02-14 19:30:08+00:00	Watsi | Fund medical treatments for people aro...	https://watsi.org/	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_2...	ANDREE N	watsi.org	Unknown	Desktop	2

IN [34]
```
mobile = ['iPhone', 'Android', 'iPad', 'Opera Mini', 'IEMobile', 'BlackBerry']
```

IN [35]
```
'iPad' in mobile
'Monty' in mobile
```

OUT [35]
```
False
```

IN [36]
```
if 'jack' in mobile:
  print ('yahoo!')
else:
    print ('WTF')
```

OUT [36]
```
WTF
```

IN [37]
```
operas = ['The Marriage of Figaro', 'The Magic Flute', 'La traviata']

if 'The Marriage of Figaro' in mobile:
  print ('grave success.')
elif 'The Marriage of Figaro' in operas:
  print ('that is a gravely beautiful piece.')
else:
  print ('that is immobile.')
```

OUT [37]
```
that is a gravely beautiful piece.
```

IN [38]
```
def is_in_mobile(platform):
    if platform in mobile:
        print ('great success.')
```

IN [39]
```
def is_in_mobile(platform):
    if platform in mobile:
      print ('great success')
    else:
      print ('Jack Black')
is_in_mobile('dog days')
```

OUT [39]
```
Jack Black
```

IN [40]
```
def filter_desktop_mobile(platform):
    if platform in mobile:
        return 'Mobile'
    elif platform == 'Desktop':
        return 'Desktop'
    else:
        return 'Not Known'
filter_desktop_mobile('BlackBerry')
```

OUT [40]
```
'Mobile'
```

IN [41]

```
what_is_ipad = filter_desktop_mobile('iPad')
what_is_ipad
```

OUT [41]

```
'Mobile'
```

IN [42]

```
print (filter_desktop_mobile('BlackBerry'))
filter_desktop_mobile('Desktop')
```

OUT [42]

```
'Desktop'

Mobile
```

IN [43]

```
print (filter_desktop_mobile('iPhone'))
print (filter_desktop_mobile('Desktop'))
print (filter_desktop_mobile('Paper Plane'))
```

OUT [43]

```
Mobile
Desktop
Not Known
```

IN [44]

```
data['platform'].apply(filter_desktop_mobile)
```

OUT [44]

```
0       Desktop
```

```
0       Desktop
1       Desktop
2       Desktop
3       Desktop
4       Desktop
        ...
4995    Desktop
4996    Desktop
4997     Mobile
4998     Mobile
4999    Desktop
Name: platform, Length: 5000, dtype: object
```

IN [45]

```
data['platform_type'] = data['platform'].apply(filter_desktop_mobile)
```

IN [46]

```
data[['platform_type','platform']][14:18]
```

OUT [46]

index	platform_type	platform
14	Desktop	Desktop
15	Mobile	Android
16	Mobile	iPhone
17	Desktop	Desktop

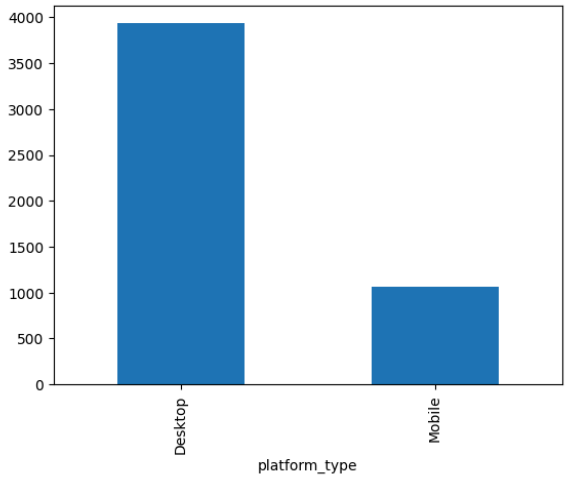
IN [47]

```
print (data['platform_type'].value_counts())
data['platform_type'].value_counts().plot(kind='bar')
```

OUT [47]

```
<Axes: xlabel='platform_type'>
```

```
platform_type
Desktop    3934
Mobile     1066
Name: count, dtype: int64
```



IN [58]

```python
def get_referrer_length(referrer):
    return len(referrer)
```

IN [59]

```python
data['referrer_len'] = data['referrer'].apply(get_referrer_length)
data[['referrer','referrer_len']].head()
```

OUT [59]

index	referrer	referrer_len
0	https://www.google.com/	23
1	https://themeteorchef.com/snippets/making-use-...	63
2	https://watsi.org/	18
3	Unknown	7
4	https://watsi.org/fund-treatments	33

IN [62]

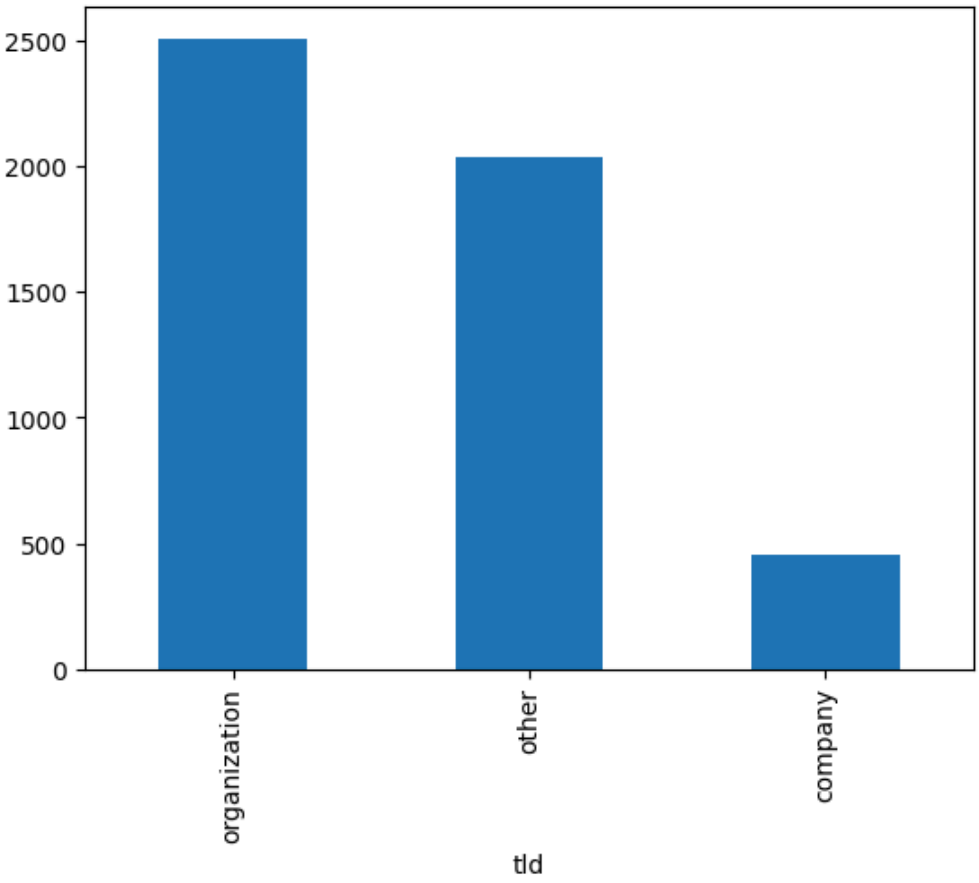
```python
tlds = ['.org','.com']
def filter_tld(domain):
  if '.org' in domain:
    return 'organization'
  elif '.com' in domain:
    return 'company'
  else:
    return 'other'
```

IN [63]

```
1  data['tld'] = data['referrer_domain'].apply(filter_tld)
2  data['tld'].value_counts().plot(kind='bar')
3
```

OUT [63]

```
<Axes: xlabel='tld'>
```

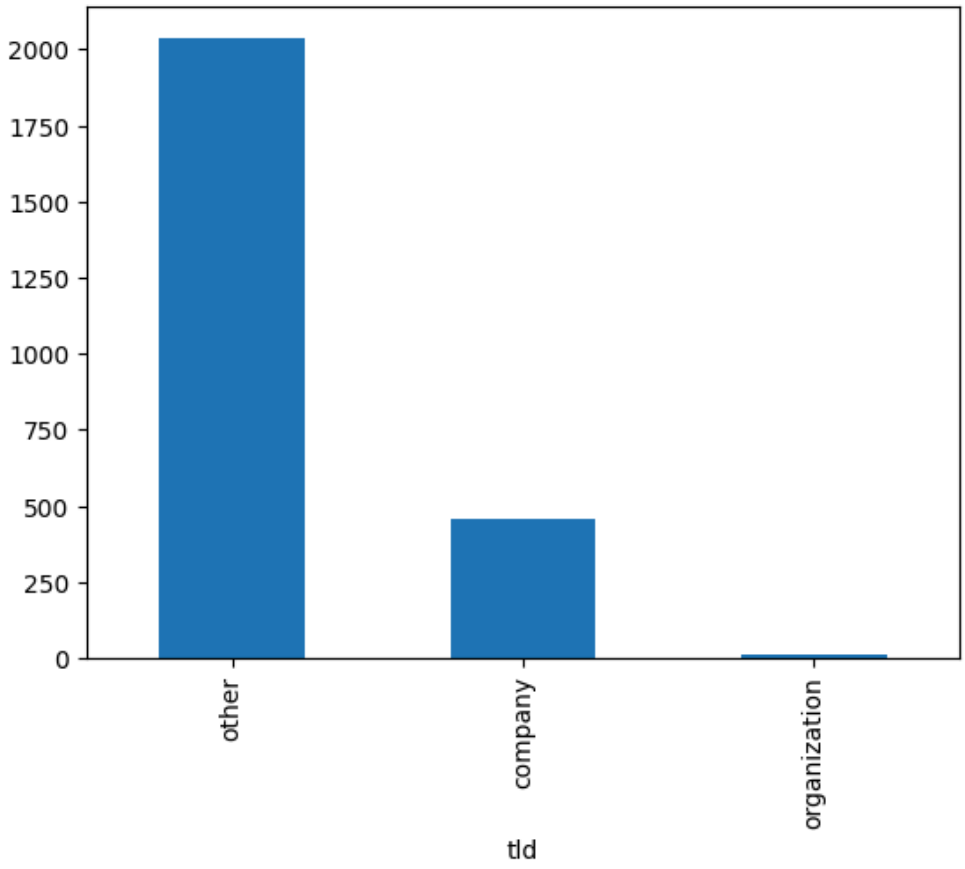


IN [64]

```
non_watsi = data[data['referrer_domain'] != 'watsi.org']
non_watsi['tld'].value_counts().plot(kind='bar')
```

OUT [64]

```
<Axes: xlabel='tld'>
```



Md

```
### Pandas .groupby(), Lambda Functions, & Pivot Tables
Sample and sort data with .sample(n=1) and .sort_values
Create Lambda functions
Group data by columns with .groupby()
Plot grouped data
Group and aggregate data with .pivot_tables()

This notebook is automatically populated with US domestic flight data&mdash;`datasets[0]`
```

IN [51]

Py

```
data[25:44]
```

OUT [51]

index	flight_date	unique_carrier	flight_num	origin	dest	arr_delay	cancelled	distance	carrier_delay	weather_delay	late_aircraft_delay	nas_delay	security_delay	actual_elapsed_ti
25	2015-01-14 00:00:00+00:00	AA	2.0	LAX	JFK	-7.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	32
26	2015-01-02 00:00:00+00:00	AA	3.0	JFK	LAX	-30.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	36
27	2015-01-03 00:00:00+00:00	AA	3.0	JFK	LAX	-14.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	36
28	2015-01-04 00:00:00+00:00	AA	3.0	JFK	LAX	-4.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
29	2015-01-05 00:00:00+00:00	AA	3.0	JFK	LAX	4.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	39
30	2015-01-06 00:00:00+00:00	AA	3.0	JFK	LAX	52.0	0.0	2475.0	0.0	0.0	0.0	52.0	0.0	44
31	2015-01-07 00:00:00+00:00	AA	3.0	JFK	LAX	-24.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	36
32	2015-01-08 00:00:00+00:00	AA	3.0	JFK	LAX	-8.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	37
33	2015-01-09 00:00:00+00:00	AA	3.0	JFK	LAX	18.0	0.0	2475.0	0.0	0.0	0.0	18.0	0.0	40
34	2015-01-10 00:00:00+00:00	AA	3.0	JFK	LAX	-2.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
35	2015-01-11 00:00:00+00:00	AA	3.0	JFK	LAX	-20.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	36
36	2015-01-12 00:00:00+00:00	AA	3.0	JFK	LAX	10.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	39
37	2015-01-13 00:00:00+00:00	AA	3.0	JFK	LAX	-30.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	36

IN [3]

Py

```
import pandas as pd
import numpy as np
```

IN [52]

Py

```
data = datasets[0]
data = data.fillna(np.nan)
```

IN [5]

Py

```
data.sample(5)
```

OUT [5]

index	flight_date	unique_carrier	flight_num	origin	dest	arr_delay	cancelled	distance	carrier_delay	weather_delay	late_aircraft_delay	nas_delay	security_delay	actual_elapsed
90407	2015-01-11 00:00:00+00:00	WN	145.0	OAK	LAX	73.0	0.0	337.0	0.0	0.0	65.0	8.0	0.0	
122902	2015-01-11 00:00:00+00:00	OO	5232.0	DEN	BIS	2.0	0.0	516.0	NaN	NaN	NaN	NaN	NaN	
23852	2015-01-03 00:00:00+00:00	B6	84.0	MCO	JFK	143.0	0.0	944.0	0.0	0.0	16.0	127.0	0.0	
23471	2015-01-02 00:00:00+00:00	B6	127.0	EWR	MCO	3.0	0.0	937.0	NaN	NaN	NaN	NaN	NaN	
191509	2015-01-06 00:00:00+00:00	MQ	3016.0	PNS	DFW	-3.0	0.0	604.0	NaN	NaN	NaN	NaN	NaN	

IN [6]

```
data.sort_values(by='arr_delay', ascending=False)[:10]
```

OUT [6]

index	flight_date	unique_carrier	flight_num	origin	dest	arr_delay	cancelled	distance	carrier_delay	weather_delay	late_aircraft_delay	nas_delay	security_delay	actual_elapsed_
11073	2015-01-11 00:00:00+00:00	AA	1595.0	AUS	DFW	1444.0	0.0	190.0	1444.0	0.0	0.0	0.0	0.0	
10214	2015-01-13 00:00:00+00:00	AA	1487.0	OMA	DFW	1392.0	0.0	583.0	1392.0	0.0	0.0	0.0	0.0	1
12430	2015-01-03 00:00:00+00:00	AA	1677.0	MEM	DFW	1384.0	0.0	432.0	1380.0	0.0	0.0	4.0	0.0	1
8443	2015-01-04 00:00:00+00:00	AA	1279.0	OMA	DFW	1237.0	0.0	583.0	1222.0	0.0	15.0	0.0	0.0	1
10328	2015-01-05 00:00:00+00:00	AA	1495.0	EGE	DFW	1187.0	0.0	721.0	1019.0	0.0	168.0	0.0	0.0	1
36570	2015-01-04 00:00:00+00:00	DL	1435.0	MIA	MSP	1174.0	0.0	1501.0	1174.0	0.0	0.0	0.0	0.0	2
36495	2015-01-04 00:00:00+00:00	DL	1367.0	ROC	ATL	1138.0	0.0	749.0	1112.0	0.0	0.0	26.0	0.0	1
59072	2015-01-14 00:00:00+00:00	DL	1687.0	SAN	MSP	1084.0	0.0	1532.0	1070.0	0.0	0.0	14.0	0.0	2
32173	2015-01-05 00:00:00+00:00	AA	970.0	LAS	LAX	1042.0	0.0	236.0	1033.0	0.0	9.0	0.0	0.0	
56488	2015-01-12 00:00:00+00:00	DL	2117.0	ATL	COS	1016.0	0.0	1184.0	1016.0	0.0	0.0	0.0	0.0	1

IN [7]

```
def is_delayed(x):
  return x>0
data['delayed']=data['arr_delay'].apply(is_delayed)
data['delayed'] = data['arr_delay'].apply(lambda x: x > 0)
data['delayed'].value_counts()
```

OUT [7]

```
delayed
False    103037
True      98627
Name: count, dtype: int64
```

IN [8]

```
data['delayed'].value_counts()
not_delayed=data['delayed'].value_counts()[0]
delayed=data['delayed'].value_counts()[1]
total_flights=not_delayed+delayed
print(int(delayed)/(total_flights))
int(not_delayed)/(total_flights)
```

OUT [8]

> 1 Warning

```
np.float64(0.5109340288797207)
```

```
0.4890659711202793
```

IN [9]

```
print (int(4/3))
print (float(4/3))
```

OUT [9]

```
1
1.3333333333333333
```

IN [10]

```
print(data['cancelled'].value_counts())
not_delayed,delayed=data['cancelled'].value_counts()
delayed/(delayed+not_delayed)
delayed/(196873+4791)
```

OUT [10]

```
0.02375733894001904
```

```
cancelled
0.0    196873
1.0      4791
Name: count, dtype: int64
```

IN [18]

```
group_by_carrier = data.groupby(['unique_carrier','delayed'])
group_by_carrier.size()
count_delays_by_carrier = group_by_carrier.size().unstack()
count_delays_by_carrier
```

OUT [18]

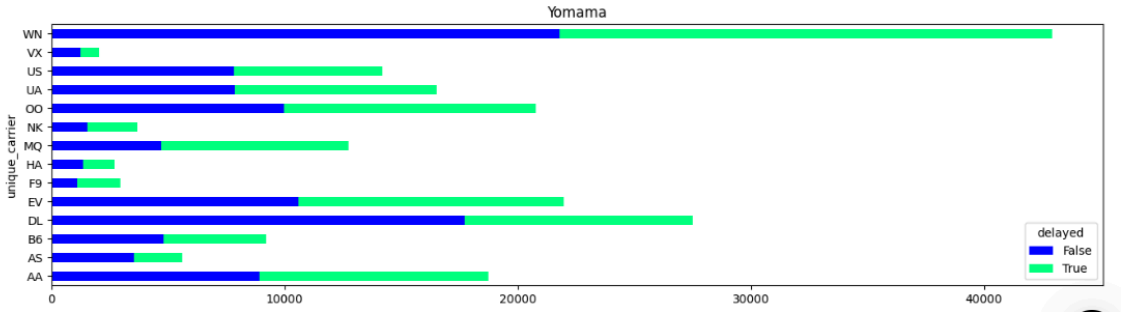
delayed	False	True
unique_carrier		
AA	8912	9841
AS	3527	2104
B6	4832	4401
DL	17719	9803
EV	10596	11371
F9	1103	1848
HA	1351	1354
MQ	4692	8060
NK	1550	2133
OO	9977	10804
UA	7885	8624
US	7850	6353
VX	1254	781
WN	21789	21150

IN [23]

```
count_delays_by_carrier.plot(kind='barh', title='Yomama', stacked=True, figsize=[16,4], colormap='winter')
```

OUT [23]

```
<Axes: title={'center': 'Yomama'}, ylabel='unique_carrier'>
```



IN [25]

```
data['delayed_20'] = data['arr_delay'].apply(lambda x: x > 20)
delay_counts = data['delayed_20'].value_counts()
delay_counts
#same as below
def is_longer_than_20(x):
    return x > 20
data['delayed_20'] = data['arr_delay'].apply(is_longer_than_20)
delay_counts = data['delayed_20'].value_counts()
delay_counts
```

OUT [25]

```
delayed_20
False    152985
True      48679
Name: count, dtype: int64
```

IN [46]

```
print 'Flights delayed longer than 20 min:', delay_counts[1]/float(delay_counts[1] + delay_counts[0]) * 100, '%'
```

OUT [46]

1 Warning

```
('Flights delayed longer than 20 min:', 201664.0)
```

IN [47]

```
flights_by_carrier = data.pivot_table(index='flight_date', columns='unique_carrier', values='flight_num', aggfunc='count')
flights_by_carrier.head()
```

OUT [47]

unique_carrier	AA	AS	B6	DL	EV	F9	HA	MQ	NK	OO	UA	US	VX	WN
flight_date														
2015-01-02 00:00:00+00:00	1545	477	759	2271	1824	254	224	1046	287	1763	1420	1177	176	3518
2015-01-03 00:00:00+00:00	1453	449	711	2031	1744	192	202	937	285	1681	1233	1028	160	3328
2015-01-04 00:00:00+00:00	1534	458	759	2258	1833	249	206	1027	284	1731	1283	1158	169	3403
2015-01-05 00:00:00+00:00	1532	433	754	2212	1811	264	209	1039	288	1737	1432	1157	174	3506
2015-01-06 00:00:00+00:00	1400	415	692	2054	1686	249	202	966	279	1527	1294	1003	152	3396

IN [48]

```
data.pivot_table(columns='flight_date',values='weather_delay')
```

OUT [48]

flight_date	2015-01-02 00:00:00+00:00	2015-01-03 00:00:00+00:00	2015-01-04 00:00:00+00:00	2015-01-05 00:00:00+00:00	2015-01-06 00:00:00+00:00	2015-01-07 00:00:00+00:00	2015-01-08 00:00:
weather_delay	1.610552	2.094342	2.142729	3.627618	4.768631	4.07876	

IN [53]

```
delays_list = ['carrier_delay','weather_delay','late_aircraft_delay','nas_delay','security_delay']
flight_delays_by_day = data.pivot_table(index='flight_date', values=delays_list, aggfunc='sum')
flight_delays_by_day
```

OUT [53]

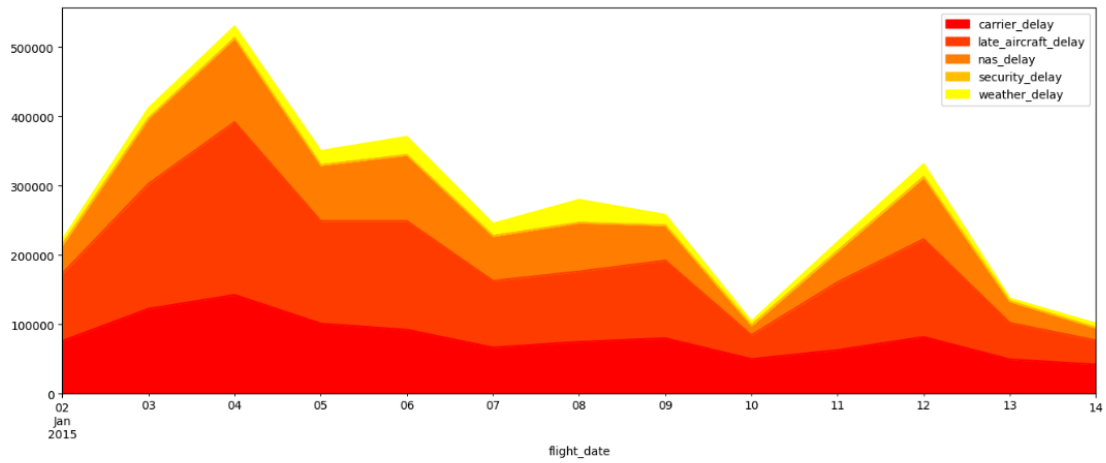
index	carrier_delay	late_aircraft_delay	nas_delay	security_delay	weather_delay
flight_date					
2015-01-02 00:00:00+00:00	76143.0	97377.0	41136.0	258.0	7357.0
2015-01-03 00:00:00+00:00	122652.0	180507.0	93791.0	950.0	14252.0
2015-01-04 00:00:00+00:00	142667.0	249595.0	121043.0	380.0	16784.0
2015-01-05 00:00:00+00:00	101335.0	148146.0	80533.0	197.0	20438.0
2015-01-06 00:00:00+00:00	92383.0	156971.0	94990.0	372.0	26299.0
2015-01-07 00:00:00+00:00	66708.0	96301.0	64607.0	165.0	17763.0
2015-01-08 00:00:00+00:00	74861.0	101703.0	70006.0	510.0	33015.0
2015-01-09 00:00:00+00:00	80123.0	112193.0	50703.0	182.0	15076.0
2015-01-10 00:00:00+00:00	50056.0	34610.0	13958.0	116.0	5683.0
2015-01-11 00:00:00+00:00	63051.0	97990.0	44853.0	164.0	12828.0
2015-01-12 00:00:00+00:00	81944.0	141282.0	89603.0	56.0	18678.0
2015-01-13 00:00:00+00:00	49557.0	52774.0	30969.0	118.0	4074.0
2015-01-14 00:00:00+00:00	42136.0	34895.0	17954.0	173.0	6176.0

IN [59]

```
print (int(flight_delays_by_day['carrier_delay'].mean()))
print (flight_delays_by_day.plot(kind='area', figsize=[16,6], stacked=True, colormap='autumn'))
```

OUT [59]

```
80278
Axes(0.125,0.11;0.775x0.77)
```

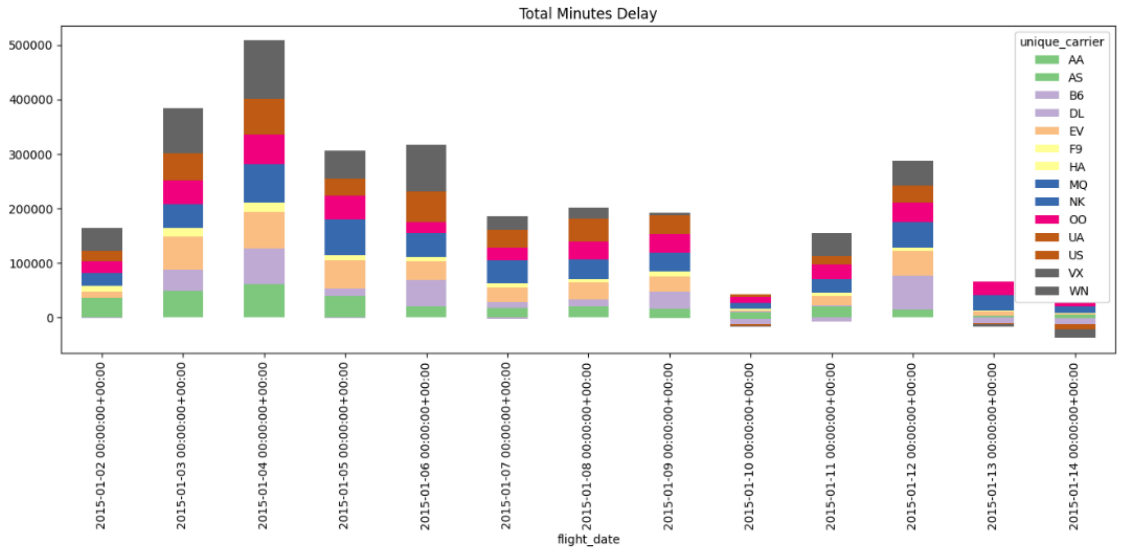


IN [68]

```
delay_by_day_carrier = data.pivot_table(index='flight_date', columns='unique_carrier', values='arr_delay', aggfunc='sum')
delay_by_day_carrier.plot(kind='bar', figsize=[16,5], stacked=True, colormap='Accent', title='Total Minutes Delay')
```

OUT [68]

```
<Axes: title={'center': 'Total Minutes Delay'}, xlabel='flight_date'>
```



Md

```
### Python Histograms, Box Plots, & Distributions
Discern the distribution of a dataset
Describe the shape of the data with basic statistics
Make histograms
Compare distributions with histograms
Make box plots

This notebook is automatically populated with US domestic flight data&mdash;`datasets[0]`
```

IN [38]

```
data.head()
```

OUT [38]

index	flight_date	unique_carrier	flight_num	origin	dest	arr_delay	cancelled	distance	carrier_delay	weather_delay	late_aircraft_delay	nas_delay	security_delay	actual_elapsed_ti
0	2015-01-02 00:00:00+00:00	AA	1.0	JFK	LAX	-19.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
1	2015-01-03 00:00:00+00:00	AA	1.0	JFK	LAX	-39.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	358
2	2015-01-04 00:00:00+00:00	AA	1.0	JFK	LAX	-12.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	385
3	2015-01-05 00:00:00+00:00	AA	1.0	JFK	LAX	-8.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	385
4	2015-01-06 00:00:00+00:00	AA	1.0	JFK	LAX	25.0	0.0	2475.0	0.0	0.0	0.0	25.0	0.0	424

IN [73]

```
import pandas as pd
import numpy as np
```

IN [40]

```
data = datasets[0]
data = data.fillna(np.nan)
```

IN [41]

```
data['delayed'] = data['arr_delay'].apply(lambda x: x > 0)
delayed_flights = data[data['delayed'] == True]
delayed_flights['unique_carrier'].value_counts()
```

OUT [41]

```
unique_carrier
WN    21150
EV    11371
OO    10804
AA     9841
DL     9803
UA     8624
MQ     8060
US     6353
B6     4401
NK     2133
AS     2104
F9     1848
HA     1354
VX      781
Name: count, dtype: int64
```

IN [42]

```
data.groupby(['unique_carrier','delayed']).size().unstack()
```

OUT [42]

delayed	False	True
unique_carrier		
AA	8912	9841
AS	3527	2104
B6	4832	4401
DL	17719	9803
EV	10596	11371
F9	1103	1848
HA	1351	1354
MQ	4692	8060
NK	1550	2133
OO	9977	10804
UA	7885	8624
US	7850	6353
VX	1254	781
WN	21789	21150

IN [43]

```
delayed_by_carrier = data.groupby(['unique_carrier','delayed']).size().unstack().reset_index()
delayed_by_carrier[:5]
```

OUT [43]

delayed	unique_carrier	False	True
0	AA	8912	9841
1	AS	3527	2104
2	B6	4832	4401
3	DL	17719	9803
4	EV	10596	11371

IN [44]

```
delayed_by_carrier['flights_count'] = (delayed_by_carrier[False] + delayed_by_carrier[True])
delayed_by_carrier[:5]
```

OUT [44]

delayed	unique_carrier	False	True	flights_count
0	AA	8912	9841	18753
1	AS	3527	2104	5631
2	B6	4832	4401	9233
3	DL	17719	9803	27522
4	EV	10596	11371	21967

IN [45]

```
delayed_by_carrier['proportion_delayed'] = delayed_by_carrier[True] / delayed_by_carrier['flights_count']
delayed_by_carrier[:4]
```

OUT [45]

delayed	unique_carrier	False	True	flights_count	proportion_delayed
0	AA	8912	9841	18753	0.524769
1	AS	3527	2104	5631	0.373646
2	B6	4832	4401	9233	0.476660
3	DL	17719	9803	27522	0.356188

IN [46]

```
1 delayed_by_carrier.sort_values('proportion_delayed', ascending=False)
```

OUT [46]

delayed	unique_carrier	False	True	flights_count	proportion_delayed
7	MQ	4692	8060	12752	0.632058
5	F9	1103	1848	2951	0.626228
8	NK	1550	2133	3683	0.579147
0	AA	8912	9841	18753	0.524769
10	UA	7885	8624	16509	0.522382
9	OO	9977	10804	20781	0.519898
4	EV	10596	11371	21967	0.517640
6	HA	1351	1354	2705	0.500555
13	WN	21789	21150	42939	0.492559
2	B6	4832	4401	9233	0.476660
11	US	7850	6353	14203	0.447300
12	VX	1254	781	2035	0.383784
1	AS	3527	2104	5631	0.373646
3	DL	17719	9803	27522	0.356188

IN [47]

```
1 data.pivot_table(columns='unique_carrier', values='arr_delay').unstack().sort_values(ascending=False)
2 #same
3 data.pivot_table(columns='unique_carrier', values='arr_delay', aggfunc=np.mean).unstack().sort_values(ascending=False)
```

OUT [47]

> 1 Warning

```
unique_carrier
MQ              arr_delay    35.627406
F9              arr_delay    28.836953
NK              arr_delay    22.779670
OO              arr_delay    19.031663
EV              arr_delay    18.358520
UA              arr_delay    16.094772
AA              arr_delay    15.616299
B6              arr_delay    13.576129
WN              arr_delay    11.273536
US              arr_delay     7.671557
HA              arr_delay     6.458937
DL              arr_delay     4.118949
VX              arr_delay     3.833908
AS              arr_delay     1.731951
dtype: float64
```

IN [48]
```
southwest = data[data['unique_carrier'] == 'WN']['arr_delay']
```

IN [49]
```
southwest.describe()
#count: there are 42,020 rows in the dataset, which is filtered to only show Southwest (WN).
#mean: the average delay.
#std: the standard deviation. More on this below.
#min: the shortest delay in the dataset. In this case, the flight was very early.
#25%: the 25th percentile. 25% of delays were lower than -9.00.
#50%: the 50th percentile, or the median. 50% of delays were lower than 1.00.
#75%: the 75th percentile. 75% of delays were lower than 19.00.
#max: the longest delay in the dataset: 535.00.
```

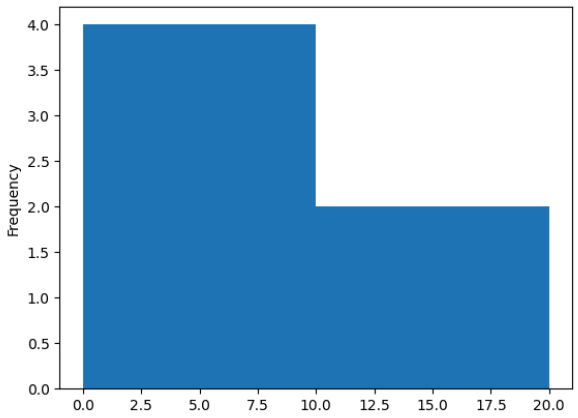
OUT [49]
```
count    42020.000000
mean        11.273536
std         36.438970
min        -55.000000
25%         -9.000000
50%          1.000000
75%         19.000000
max        535.000000
Name: arr_delay, dtype: float64
```

IN [50]
```
data[data['origin'] == 'ORD']['arr_delay'].describe()
```

OUT [50]
```
count    9207.000000
mean       38.073205
std        63.261760
min       -49.000000
25%        -3.000000
50%        17.000000
75%        57.000000
max       751.000000
Name: arr_delay, dtype: float64
```

IN [51]
```
#histogram
A = [0,10]
B = [10,20]
[1, 2, 5, 9, 12, 20] # our data
pd.Series([1, 2, 5, 9, 12, 20]).plot(kind='hist', bins=[0,10,20]) # `bins` defines the start and end points of bins
```

OUT [51]
```
<Axes: ylabel='Frequency'>
```



IN [52]
```
bin_values = np.arange(start=-50, stop=200, step=10)
bin_values
```

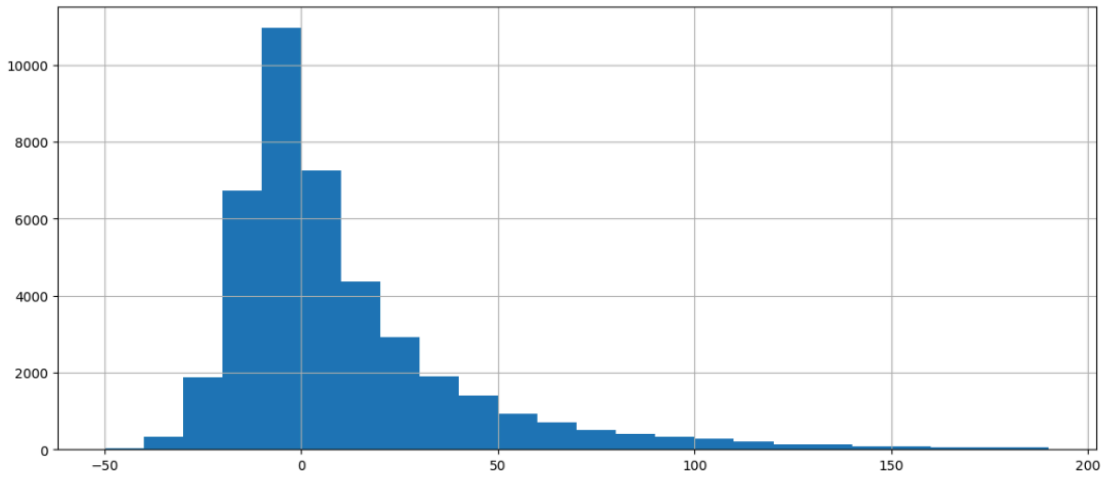
OUT [52]
```
array([-50, -40, -30, -20, -10,   0,  10,  20,  30,  40,  50,  60,  70,
        80,  90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190])
```

IN [53]

```
wn_carrier = data[data['unique_carrier'] == 'WN']
wn_carrier['arr_delay'].hist(bins=bin_values, figsize=[14,6])
```

OUT [53]

```
<Axes: >
```

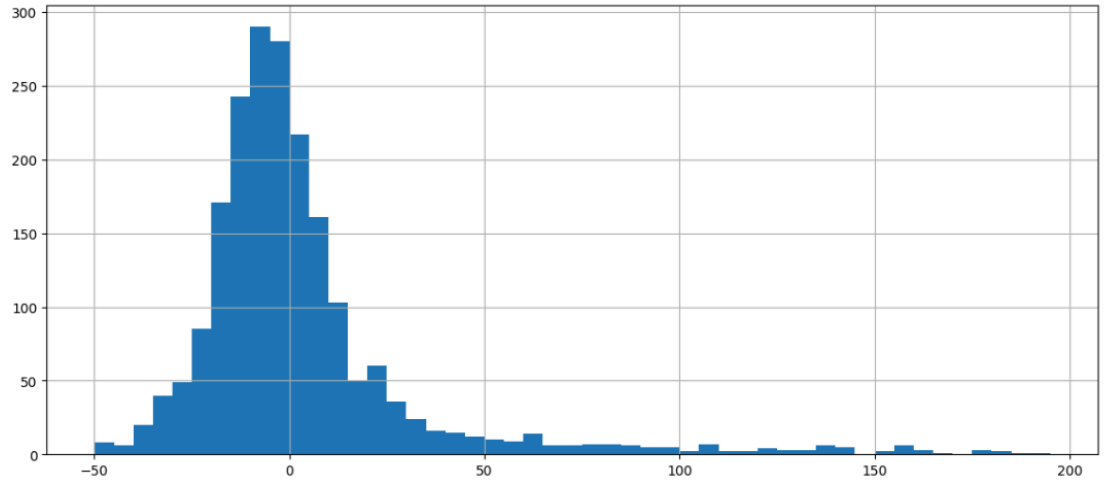


IN [54]

```
# another example
virgin_bin_values = np.arange(start=-50, stop=200, step=5)
virgin_carrier = data[data['unique_carrier'] == 'VX']
virgin_carrier['arr_delay'].hist(bins=virgin_bin_values, figsize=[14,6])
```

OUT [54]

```
<Axes: >
```



IN [55]

```
virgin_carrier.head()
filter_criterion = (virgin_carrier['arr_delay'] > 20) & (virgin_carrier['arr_delay'] < 25)
virgin_late_20 = virgin_carrier[filter_criterion]
print (virgin_late_20['origin'].shape)
print (virgin_late_20['origin'].value_counts())
```

OUT [55]

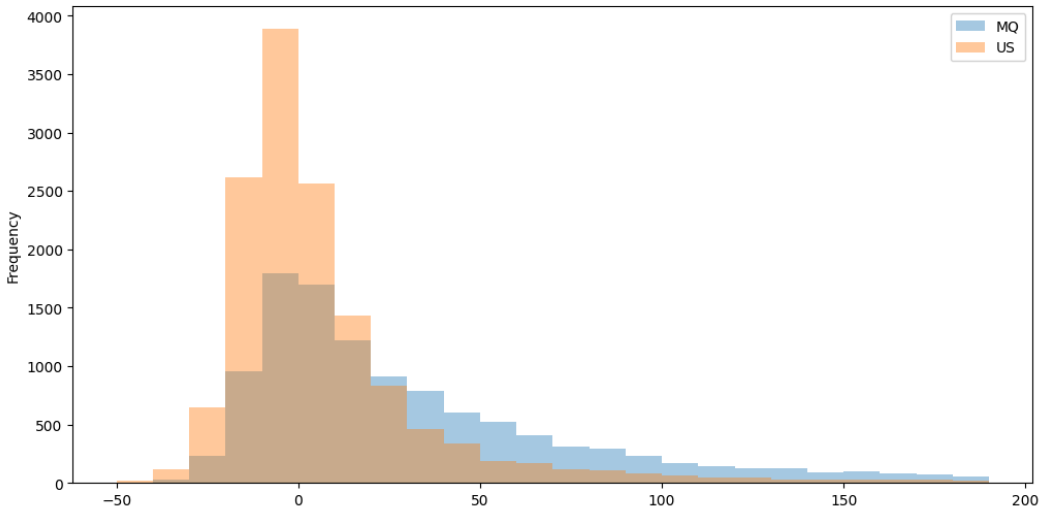
```
(48,)
origin
SFO    13
LAX    12
JFK     4
EWR     4
DAL     4
IAD     3
FLL     2
ORD     2
BOS     1
LAS     1
LGA     1
DCA     1
Name: count, dtype: int64
```

IN [56]

```
bin_values = np.arange(start=-50, stop=200, step=10)
us_mq_airlines_index = data['unique_carrier'].isin(['US','MQ']) # create index of flights from those airlines
us_mq_airlines = data[us_mq_airlines_index] # select rows
group_carriers = us_mq_airlines.groupby('unique_carrier')['arr_delay'] # group values by carrier, select minutes delayed
group_carriers.plot(kind='hist', bins=bin_values, figsize=[12,6], alpha=.4, legend=True) # alpha for transparency
```

OUT [56]

```
unique_carrier
MQ    Axes(0.125,0.11;0.775x0.77)
US    Axes(0.125,0.11;0.775x0.77)
Name: arr_delay, dtype: object
```



IN [57]

```
group_carriers.describe()
```

OUT [57]

index	count	mean	std	min	25%	50%	75%	max
unique_carrier								
MQ	11275.0	35.627406	58.444090	-51.0	-2.0	17.0	53.0	788.0
US	13972.0	7.671557	34.672795	-59.0	-10.0	-1.0	13.0	621.0

IN [80]

```
print('US Airways', group_carriers.describe().loc['US']['std'], 'minutes from the mean', group_carriers.describe().loc['US']['mean'])
print('Envoy Air', group_carriers.describe().loc['MQ']['std'], 'minutes from the mean', group_carriers.describe().loc['MQ']['mean'])
```

OUT [80]

```
US Airways 34.67279466146994 minutes from the mean 7.671557400515316
Envoy Air 58.44409039663552 minutes from the mean 35.62740576496674
```

IN [59]

```
hi_volume = data['origin'].value_counts()[:20]
hi_volume
hi_volume_airports_names = hi_volume.index.tolist()
hi_volume_airports_names
```

OUT [59]

```
['ATL',
 'ORD',
 'DFW',
 'DEN',
 'LAX',
 'IAH',
 'PHX',
 'SFO',
 'LAS',
 'MCO',
 'LGA',
 'DTW',
 'CLT',
 'MSP',
 'EWR',
 'SLC',
 'BOS',
 'SEA',
 'JFK',
 'FLL']
```

IN [60]

```
hi_volume_airports = data[data['origin'].isin(hi_volume_airports_names)]
hi_volume_airports.head()
```

OUT [60]

index	flight_date	unique_carrier	flight_num	origin	dest	arr_delay	cancelled	distance	carrier_delay	weather_delay	late_aircraft_delay	nas_delay	security_delay	actual_elapsed_ti
0	2015-01-02 00:00:00+00:00	AA	1.0	JFK	LAX	-19.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
1	2015-01-03 00:00:00+00:00	AA	1.0	JFK	LAX	-39.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	35
2	2015-01-04 00:00:00+00:00	AA	1.0	JFK	LAX	-12.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
3	2015-01-05 00:00:00+00:00	AA	1.0	JFK	LAX	-8.0	0.0	2475.0	NaN	NaN	NaN	NaN	NaN	38
4	2015-01-06 00:00:00+00:00	AA	1.0	JFK	LAX	25.0	0.0	2475.0	0.0	0.0	0.0	25.0	0.0	42

IN [61]

```
hi_volume_airports_pivots = hi_volume_airports.pivot_table(index='flight_date', columns='origin', values='arr_delay')
hi_volume_airports_pivots
```

OUT [61]

origin	ATL	BOS	CLT	DEN	DFW	DTW	EWR	FLL	IAH	JFK	LAS	LAX	LGA	MCO
flight_date														
2015-01-02 00:00:00+00:00	3.327536	3.590580	0.509317	20.526899	36.049598	-6.842809	8.316993	-0.543307	12.156187	3.688742	13.709512	16.500000	-2.947712	5.834734
2015-01-03 00:00:00+00:00	15.428112	30.471616	13.768340	51.186292	37.604138	22.738007	37.370229	15.666667	39.844037	31.882979	18.550685	26.117338	15.606426	17.511364
2015-01-04 00:00:00+00:00	21.423343	26.867857	23.325077	52.495238	38.360104	35.771626	53.617978	25.293651	20.464286	55.445578	19.564767	28.159016	32.450704	39.847025
2015-01-05 00:00:00+00:00	3.095000	11.208609	6.051672	29.899200	28.705263	24.696594	22.674051	13.711864	8.450505	19.554422	17.229381	15.788618	34.984177	14.929204
2015-01-06 00:00:00+00:00	6.361725	43.310580	13.294964	15.344029	11.534626	35.078616	43.104530	23.425926	3.622642	43.359073	13.330579	7.234004	61.165049	29.996785
2015-01-07 00:00:00+00:00	0.944276	10.651316	4.869565	33.301095	10.428762	13.403727	22.030508	11.254464	10.490476	15.536680	7.498652	5.442446	46.063973	8.977918
2015-01-08 00:00:00+00:00	3.033099	6.807692	10.484568	14.569873	11.217450	20.593060	15.419463	2.558442	1.571121	2.749091	8.597911	6.171329	3.575221	9.152648
2015-01-09 00:00:00+00:00	1.833499	21.045603	5.742331	21.551237	8.591810	34.665653	22.632107	1.808696	7.611354	43.294964	4.487245	8.144112	42.325581	8.758410
2015-01-10 00:00:00+00:00	-5.473046	3.763547	-1.658915	2.822014	5.501582	2.584906	0.422680	-5.172269	0.937888	1.259259	2.564706	2.709746	-11.311475	0.273273
2015-01-11 00:00:00+00:00	-2.118085	-2.569767	5.789286	16.045977	19.767313	5.808725	-1.670543	-3.008734	17.064904	-2.964158	40.793103	24.195531	-7.576923	-2.242991
2015-01-12 00:00:00+00:00	42.375375	8.254777	14.975524	22.791444	19.114820	24.692771	8.219780	8.960699	22.710526	4.297101	12.710526	10.982175	16.641509	21.563863
2015-01-13 00:00:00+00:00	2.812957	-9.384106	0.086505	9.789279	7.248656	-2.710692	-2.901024	-7.118721	1.415274	-13.214559	-2.937853	-1.553506	-0.883234	-1.462295
2015-01-14 00:00:00+00:00	-1.400000	-3.091216	-1.681250	-0.638838	2.690160	-1.903727	-5.456446	3.360360	-0.530120	-14.911877	-3.695418	-2.958559	0.002994	1.885350

IN [65]

```
1 hi_volume_airports_pivots.describe()
```

OUT [65]

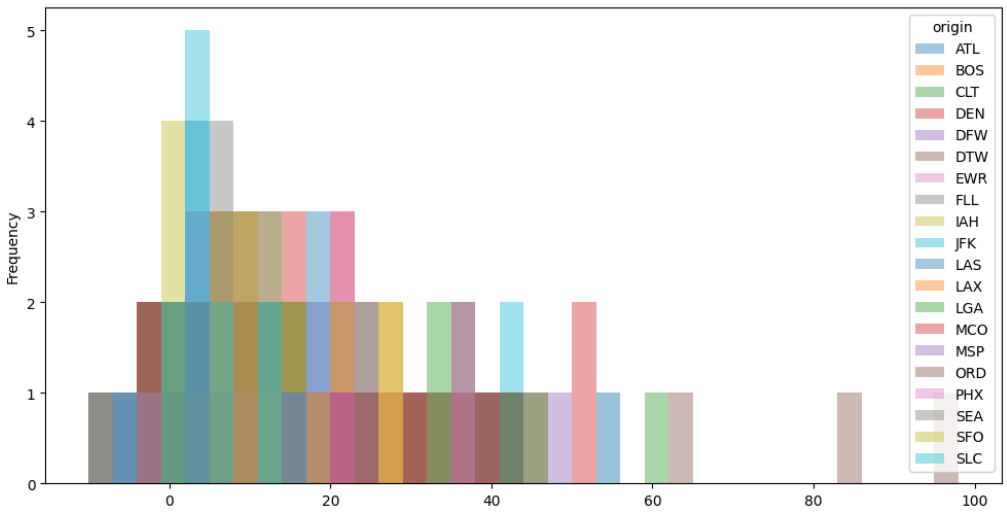
origin	ATL	BOS	CLT	DEN	DFW	DTW	EWR	FLL	IAH	JFK	LAS	LAX	LGA	MCO	MSP
count	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000	13.000000
mean	7.049522	11.609776	7.350537	22.283364	18.216483	16.044343	17.213870	6.938287	11.216083	14.613638	11.723369	11.302481	17.699715	11.925022	9.627240
std	12.798122	15.004838	7.499172	16.171575	12.854437	15.286101	18.718574	10.452380	11.488504	22.619487	11.574100	10.193057	23.428830	12.647029	14.971524
min	-5.473046	-9.384106	-1.681250	-0.638838	2.690160	-6.842809	-5.456446	-7.118721	-0.530120	-14.911877	-3.695418	-2.958559	-11.311475	-2.242991	-7.691030
25%	0.944276	3.590580	0.509317	14.569873	8.591810	2.584906	0.422680	-0.543307	1.571121	1.259259	4.487245	5.442446	-0.883234	1.885350	1.193333
50%	3.033099	8.254777	5.789286	20.526899	11.534626	20.593060	15.419463	3.360360	8.450505	4.297101	12.710526	8.144112	15.606426	8.977918	6.834459
75%	6.361725	21.045603	13.294964	29.899200	28.705263	24.696594	22.674051	13.711864	17.064904	31.882979	17.229381	16.500000	34.984177	17.511364	19.461279
max	42.375375	43.310580	23.325077	52.495238	38.360104	35.771626	53.617978	25.293651	39.844037	55.445578	40.793103	28.159016	61.165049	39.847025	47.264605

IN [76]

```
1 airport_bins = np.arange(-10,100,3)
2 hi_volume_airports_pivots.plot(kind='hist', bins=airport_bins, figsize=[12,6], alpha=.4, legend=True)
```

OUT [76]

```
<Axes: ylabel='Frequency'>
```

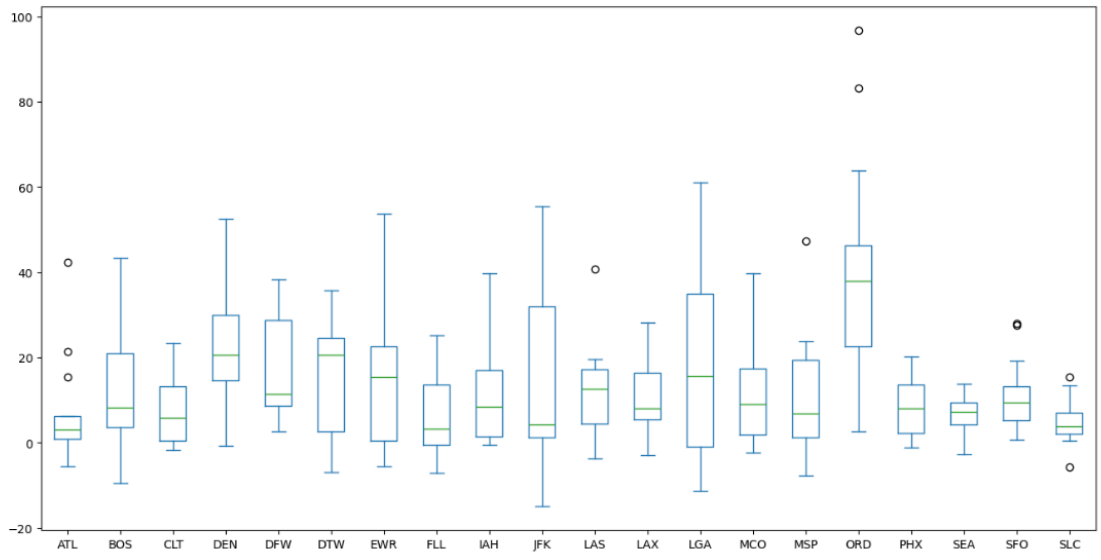


IN [81]

```
hi_volume_airports_pivots = hi_volume_airports.pivot_table(index='flight_date', columns='origin', values='arr_delay')
hi_volume_airports_pivots.plot(kind='box', figsize=[16,8])
```

OUT [81]

<Axes: >



IN [84]

```
hi_volume_airports.pivot_table(
    index='flight_date',
    columns='unique_carrier',
    values='arr_delay'
).plot(kind='box')
```

OUT [84]

```
<Axes: >
```

